

## Foundation of Blockchain Technology (IS501) Pre-Reading Material – Lecture 18

### Solidity Data Modifiers Part-2

**Estimated Reading Time:** 15–20 minutes

### Learning Objectives

After completing this pre-reading, students will be able to:

- Explain the purpose of view and pure modifiers in Solidity.
- Differentiate between read-only and state-changing functions.
- Identify getter and setter functions in a smart contract.
- Understand the relationship between state changes and gas consumption.
- Distinguish compiler errors from semantic errors.
- Understand how Solidity functions can return multiple values.

### Introduction to Data Modifiers

In Solidity, **data/behavioral modifiers** describe how a function interacts with the blockchain state.

The three important categories are:

| Modifier           | Reads State | Modifies State | Typical Purpose           |
|--------------------|-------------|----------------|---------------------------|
| <b>view</b>        | Yes         | No             | Reading stored data       |
| <b>pure</b>        | No          | No             | Local calculations        |
| <b>No modifier</b> | May         | May            | State-changing operations |

**Key idea:** view and pure restrict what a function is allowed to do.

Recap: Visibility Modifiers

Before understanding data modifiers, recall the four main **visibility modifiers**:

- **public** – accessible from inside and outside the contract.
- **private** – accessible only within the same contract.
- **internal** – accessible within the contract and derived contracts.
- **external** – generally called from outside the contract.

Prepared by: Dr. Rajesh Kumar

Foundation of Blockchain Technology (IS501) | AY 2026–27

Course Repository: <https://github.com/rajesh101191/Foundation-of-Blockchain-Technology>



### Example

```
function getData() public view returns (uint) {  
    return id;  
}
```

Here:

- public → determines **who can call the function**.
- view → determines **what the function can do with blockchain state**.

Thus, **visibility and data modifiers serve different purposes**.

### The view Modifier

The view modifier tells Solidity that a function **can read blockchain state but cannot modify it**.

### Example

```
function getStateVariables() public view returns (uint, string memory) {  
    return (id, name);  
}
```

The function reads the values of id and name, but does not change them.

### Characteristics of view

- Can read state variables.
- Cannot modify state variables.
- Cannot perform operations that modify blockchain state.
- Useful for getter functions.
- An external read-only call normally does **not require the caller to pay gas**.

### Example of an invalid operation

```
function changeData() public view {  
    id = 10;  
}
```

This produces a **compiler error** because a view function is not allowed to modify id.

## The pure Modifier

The pure modifier is even more restrictive than view.

A pure function:

- Does not read blockchain state.
- Does not modify blockchain state.
- Performs calculations using its parameters or local variables.

### Example

```
function add(uint a, uint b) public pure returns (uint) {
    return a + b;
}
```

The function does not access any stored contract variable. It only works with the values supplied as inputs.

### view vs pure

| Feature                | view                | pure         |
|------------------------|---------------------|--------------|
| Read state variables   | ✓Yes                | ✗No          |
| Modify state variables | ✗No                 | ✗No          |
| Uses input/local data  | ✓Yes                | ✓Yes         |
| Typical use            | Reading stored data | Calculations |
| Easy way to remember   |                     |              |

**view = Look but don't change**

**pure = Don't look and don't change**

## State-Changing Functions

A function without view or pure may modify blockchain state.

Prepared by: Dr. Rajesh Kumar

Foundation of Blockchain Technology (IS501) | AY 2026–27

Course Repository: <https://github.com/rajesh101191/Foundation-of-Blockchain-Technology>



For example:

```
function setbMsg(bytes32 newValue) private {  
    bMsg = newValue;  
}
```

Here, bMsg is changed.

Therefore, when such a function is executed as a blockchain transaction:

- The state is updated.
- The transaction requires gas.
- The updated state is stored on the blockchain.

### Important distinction

Not every function without view or pure **necessarily changes state every time it executes**. Rather, it is **allowed to perform state-changing operations**.

## Getter and Setter Functions

### Getter

A getter is used to **retrieve information** from a contract.

Example:

```
function getStateVariables() public view returns (uint, string memory) {  
    return (id, name);  
}
```

It reads stored values without changing them.

### Setter

A setter is used to **update information** stored in the contract.

```
function setbMsg(bytes32 newValue) private {  
    bMsg = newValue;  
}
```



The setter changes the value of bMsg.

### Getter vs Setter

| Feature                                      | Getter    | Setter       |
|----------------------------------------------|-----------|--------------|
| <b>Purpose</b>                               | Read data | Update data  |
| <b>Usually uses</b>                          | view      | No view/pure |
| <b>Changes state</b>                         | No        | Yes          |
| <b>Transaction required for state change</b> | No        | Yes          |
| <b>Gas for blockchain transaction</b>        | No        | Yes          |

**Note:** A public state variable automatically gets a compiler-generated getter in Solidity. A separate getter function is not always necessary.

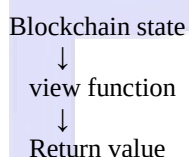
## Understanding Gas

**Gas** is the unit used to measure the computational work required to execute operations on Ethereum-compatible blockchains.

A useful distinction is:

### Reading

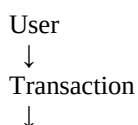
When a view function is called externally as an ordinary read:



There is generally **no transaction fee for the user** because no transaction is being submitted to modify the blockchain.

### Writing

For a state-changing function:



Prepared by: Dr. Rajesh Kumar

Foundation of Blockchain Technology (IS501) | AY 2026–27

Course Repository: <https://github.com/rajesh101191/Foundation-of-Blockchain-Technology>



Smart Contract



State modified



Gas consumed

The transaction requires gas because the blockchain must execute and record the state change.

## Multiple Return Values

Solidity allows a function to return more than one value.

Example:

```
function getStateVariables()  
    public  
    view  
    returns (uint, string memory)  
{  
    return (id, name);  
}
```

Here, the function returns:

1. id
2. name

This is useful when several related values need to be retrieved together.

## Compiler Errors and Semantic Errors

Understanding errors is important before deploying smart contracts.

### Compiler Error

A compiler error is detected **before the contract can be successfully compiled**.

Example:



```
function update() public view {
    id = 100;
}
```

The compiler identifies that a view function is attempting to modify state.

### Why compiler errors are useful

They prevent invalid code from being compiled and help identify mistakes before deployment.

### Semantic/Logic Error

A semantic or logic error occurs when the code is syntactically valid and compiles, but the program does not behave as intended.

Example:

```
function calculate(uint a, uint b)
    public
    pure
    returns (uint)
{
    return a - b;
}
```

The function may compile successfully, but if the intended operation was addition, the logic is incorrect.

### Comparison

| Error                       | Detected                 | Example                     |
|-----------------------------|--------------------------|-----------------------------|
| <b>Compiler error</b>       | During compilation       | Modifying state inside view |
| <b>Logic/Semantic error</b> | During testing/execution | Incorrect calculation       |

**Key point:** Successful compilation does **not** guarantee correct program behavior.

## Connecting the Concepts

Consider the following contract:

```
contract Data Type {  
  
    uint id;  
    string name;  
  
    function getStateVariables()  
        public  
        view  
        returns (uint, string memory)  
    {  
        return (id, name);  
    }  
  
    function setId(uint newId) private {  
        id = newId;  
    }  
  
    function add(uint a, uint b)  
        public  
        pure  
        returns (uint)  
    {  
        return a + b;  
    }  
}
```

The three functions have different responsibilities:

| Function                   | Visibility | Data Modifier | Purpose             |
|----------------------------|------------|---------------|---------------------|
| <b>getStateVariables()</b> | public     | view          | Read stored data    |
| <b>setId()</b>             | private    | None          | Modify stored data  |
| <b>add()</b>               | public     | pure          | Perform calculation |

This illustrates an important concept:

**Visibility controls access, while view/pure control interaction with contract state.**



## Quick Concept Check

Before attending the lecture, try answering these questions:

1. What is the difference between a visibility modifier and a data modifier?
2. Can a view function modify a state variable?
3. Can a pure function read a state variable?
4. Why do state-changing transactions consume gas?
5. What is the difference between a getter and a setter?
6. Why would the compiler reject a state modification inside a view function?
7. What is the main purpose of a pure function?
8. Can a Solidity function return multiple values?
9. Does successful compilation guarantee that a smart contract is logically correct?
10. What is the difference between public view and public pure?

## Key Takeaways

- **Visibility modifiers** determine **who can access a function**.
- **view** functions can read state but cannot modify it.
- **pure** functions neither read nor modify contract state.
- **State-changing functions** can modify blockchain data and require gas when executed through a transaction.
- **Getters** retrieve data; **setters** update data.
- External calls to view/pure functions are generally read-only and do not require the caller to pay a transaction gas fee.
- Solidity functions can return **multiple values**.
- Compiler errors are detected before successful compilation, while logic errors can remain even after compilation.
- Minimizing unnecessary state changes can reduce transaction costs.

## One-line memory aid

**public/private** → **Who can access?** | **view** → **Read state** | **pure** → **No state access** | **No data modifier** → **May change state**

